SYMBIOSIS INSTITUTE OF TECHNOLOGY, HYDERABAD
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

PROJECT REPORT

ON

"LIBRARY MANAGEMENT SYSTEM:

Data Structures Based Book, Member and Transaction Manager"

Subject: Data Structures - Lab

Submitted By:

Name	Branch	PRN
Dharman Khambhalia	CSE	25070721014
Anshuman Yadav	CSE	25070721007

INDEX

1. Acknowledgement
2. Abstract and Introduction
3. Objectives
4. Methodology
5. Data Structures Used
6. Features and Rules
7. Complexity Analysis
8. Applications and Future Scope
9. Output Screenshots

ACKNOWLEDGEMENT

We take this opportunity to express our heartfelt gratitude to everyone who has guided and supported us throughout the completion of our project titled *"LIBRARY MANAGEMENT SYSTEM: Data Structures Based Book, Member and Transaction Manager."*

First and foremost, we extend our deepest thanks to **Dr. Rakesh Salakapuri**, our professor and project guide, for his constant encouragement, valuable guidance, and continuous support. The insightful feedback and in-depth knowledge helped us understand how to choose the right data structure for each task and bring our concept to life as a fully functional program. This mentorship has been a constant source of inspiration throughout this journey.

We would like to express our sincere gratitude to Dr. Rajinikanth Aluvalu, Director of Symbiosis Institute of Technology, Hyderabad, for providing us with the necessary infrastructure, resources, and an environment that encourages innovation, experimentation, and research. His vision for academic excellence has motivated us to pursue this project with dedication and creativity.

Finally, we would like to express our gratitude to our peers and classmates for their helpful suggestions, cooperation, and moral support throughout the project. We are also thankful to Symbiosis Institute of Technology, Hyderabad, for providing us with the platform and resources to transform our ideas into reality.

ABSTRACT AND INTRODUCTION

Library Management System is a menu-driven C program in which every feature is backed by the data structure best suited to it. A **binary search tree** holds the book catalogue, a **hash table** holds the members, **linked lists** hold the books issued to each member, **queues** hold the waitlist for each book, and a **stack** records transactions so that the last one can be undone. **Merge sort** and tree traversal add a title-sorted view and keyword search.

A library repeats a small set of tasks all day: find a book, find a member, issue, return, reserve, and correct a mistake. A single array or list can perform all of them, but slowly and awkwardly. Matching each task to a suitable data structure makes the code simpler and the operations faster.

This project applies the concepts taught in the Data Structures course to one practical application. It also shows that the **choice of data structure**, not just the algorithm, decides how well a program behaves.

The approach supports learning and demonstration of **core data structures**, and can be extended into a complete **library automation** tool with persistent storage and a graphical interface.

OBJECTIVES

- Store and search the book catalogue efficiently using a **binary search tree**.
- Look up members in near-constant time using a **hash table with chaining**.
- Track issued books with **linked lists** and fair waitlists with **queues**.
- Record every issue and return on a **stack** and support **undo**.
- Present the catalogue sorted by title using **merge sort**.
- Enforce real library rules such as a three-book limit and first-come, first-served reservations.

METHODOLOGY

1. Identify the Tasks:

- List what a library does: add or remove a book, register a member, issue, return, reserve, undo, search and list.

2. Choose a Structure for Each Task:

- Pick the structure whose strengths match the task, e.g. a queue for waitlists because the first person to reserve must be served first.

3. Define the Records:

- Book node: ID, title, author, copies, waitlist and two child links.
- Member node: ID, name, loan list and chain link. Transaction node: type, member ID and book ID.

4. Link Records by ID:

- Books and members never point at each other. Loans store book IDs and waitlists store member IDs, which keeps deletion and undo simple.

5. Write the Operations and Rules:

- Implement each operation with the checks around it, such as the loan limit and the waitlist order.

6. Build the Menu and Test:

- Wrap everything in a numbered menu, preload seed data (7 books and 3 members: Arjun, Ram, Ravi) and test a scenario that exercises every structure.
- This program has been developed using the C programming language and compiled with gcc. It uses dynamic memory and frees all of it on exit.

DATA STRUCTURES USED

Seven techniques are used, each for a different job.

1. Binary Search Tree: the book catalogue

Each book is a tree node keyed by its **book ID**. Smaller IDs go to the left child and larger IDs to the right. Search follows a single path from the root. Deleting a node with two children replaces it with the smallest ID in its right subtree, and an inorder traversal lists all books in ID order. Duplicate IDs are rejected.

2. Hash Table with Chaining: members

The bucket for a member is the **member ID mod 11**. Members whose IDs collide are chained in the same bucket, with the newest at the head. In the seed data, Arjun (ID 1) and Ravi (ID 12) both land in bucket 1, and Ram (ID 2) is in bucket 2.

3. Singly Linked List: books issued to a member

Each member owns a list of the book IDs they hold. A new loan is added at the head, and a return unlinks the book from wherever it sits in the chain. A member may hold at most three books.

4. Queue: waitlist for each book

Every book has its own **first-in, first-out** queue of member IDs, built from a linked list with front and rear pointers. Reserving adds a member at the rear. When a copy is returned, the member at the front is named as next, and nobody else can issue that copy.

5. Stack: history and undo

Every successful issue or return is pushed onto a stack. **Undo** pops the top entry and reverses it, which suits a last-in, first-out structure. Undoing a return is refused if no copy is free or the member is already at the loan limit. The history view reads the stack from newest to oldest.

6. Merge Sort: catalogue by title

The tree is copied into an array of book pointers in ID order, then sorted by title with a case-insensitive merge sort. Merge sort is stable and always takes O(n log n) time, even when the tree itself is unbalanced.

7. Tree Traversal: keyword search

A keyword search walks the tree in order and prints every book whose title or author contains the keyword, ignoring case.

FEATURES AND RULES

The program offers 14 menu options, each tied to one data structure.

Option	Feature	Data structure
1	Add book	Binary search tree
2	Remove book	Binary search tree
3	Search by ID	Binary search tree
4	Search by keyword	Tree traversal
5	List all books by ID	Inorder traversal
6	List books sorted by title	Merge sort
7	View waitlist	Queue
8	Register member	Hash table
9	View member and issued books	Hash table, linked list
10	Issue book	Linked list, queue, stack
11	Return book	Linked list, stack
12	Reserve book	Queue
13	Undo last transaction	Stack
14	Transaction history	Stack

Rules enforced by the system

- A member can hold at most 3 books.
- A member cannot issue the same book twice.
- A book with no free copy cannot be issued; the member must reserve it.
- When a book has a waitlist, the returned copy is held for the member at the front of the queue.
- A book cannot be removed while any copy is issued.
- Duplicate book IDs and member IDs are rejected; invalid input is reported instead of crashing.

COMPLEXITY ANALYSIS

Most operations are O(1) or O(log n) on average; the only slow case is an unbalanced tree. Here n is the number of books and m is the number of members.

Operation	**Structure**	**Average**	**Worst case**
Add, find or remove a book	Binary search tree	O(log n)	O(n)
Find or add a member	Hash table	O(1)	O(m)
Add or remove a loan	Linked list ($k \leq 3$)	O(1) / O(k)	O(k)
Reserve, dequeue	Queue	O(1)	O(1)
Issue, return, undo (stack step)	Stack	O(1)	O(1)
List sorted by title	Merge sort	O(n log n)	O(n log n)
Keyword search	Tree traversal	O(n)	O(n)

The tree is not self-balancing. If book IDs are inserted in sorted order, it degrades to a list and its operations become O(n). Merge sort needs O(n) extra space for its temporary array.

APPLICATIONS AND FUTURE SCOPE

Applications

1. A college or school library that manages a small catalogue and a few hundred members.
2. A teaching tool that shows how seven data structures cooperate in one program.
3. The core of a tool for other lending systems, such as equipment, laptop or lab-kit booking.
4. A base for experimenting with fairness rules such as priority queues for waitlists.

Future Scope

- Saving and loading the catalogue, members and loans from a file so data **survives between runs**.
- Adding **due dates and fines** for late returns.
- Replacing the plain tree with a self-balancing **AVL or red-black tree** to guarantee O(log n) search.
- Adding a **graphical or web front end**.
- Supporting search by author or subject using additional indexes.

OUTPUT SCREENSHOTS

```
Main menu

================ LIBRARY MANAGEMENT SYSTEM ================
  BOOKS                          MEMBERS
   1. Add book                    8. Register member
   2. Remove book                 9. View member / issued books
   3. Search by ID  (BST)
   4. Search by keyword           TRANSACTIONS
   5. List all (by ID, inorder)  10. Issue book
   6. List sorted by title       11. Return book
      (merge sort)               12. Reserve book (queue)
   7. View waitlist              13. Undo last transaction (stack)
                                 14. Transaction history
   0. Exit
============================================================
```

(this is the main menu of the program; each option is tied to one data structure)

```
List all books: by ID (BST inorder), then by title (merge sort)

  ID    Title                          Author                Avail  Waiting
  ------------------------------------------------------------------------
  101   Clean Code                     Robert Martin         1/1    0
  102   The C Programming Language     Kernighan, Ritchie    1/1    0
  104   Database System Concepts       Korth                 1/1    0
  105   Introduction to Algorithms     Cormen et al.         2/2    0
  108   Discrete Mathematics           Kenneth Rosen         2/2    0
  110   Data Structures Using C        Reema Thareja         3/3    0
  112   Operating System Concepts      Silberschatz          2/2    0
  ID    Title                          Author                Avail  Waiting
  ------------------------------------------------------------------------
  101   Clean Code                     Robert Martin         1/1    0
  110   Data Structures Using C        Reema Thareja         3/3    0
  104   Database System Concepts       Korth                 1/1    0
  108   Discrete Mathematics           Kenneth Rosen         2/2    0
  105   Introduction to Algorithms     Cormen et al.         2/2    0
  112   Operating System Concepts      Silberschatz          2/2    0
  102   The C Programming Language     Kernighan, Ritchie    1/1    0
Goodbye!
```

(books listed by ID using the tree's inorder traversal, then by title using merge sort)

```
Issue, reserve, waitlist, return, undo and history

  Issued "The C Programming Language" to Arjun.
  No copies available. Use 'Reserve' to join the waitlist.
  Ram added to waitlist for "The C Programming Language" (position 1).
  Ravi added to waitlist for "The C Programming Language" (position 2).
  Waitlist for "The C Programming Language":
    1. Ram (ID 2)
    2. Ravi (ID 12)
  "The C Programming Language" returned by Arjun.
  Notify: Ram (ID 2) is next in the waitlist.
  Copy is held for the next member in the waitlist (ID 2).
  Undid RETURN of "The C Programming Language" by Arjun.
  Most recent first:
   1. ISSUE   Arjun          The C Programming Language
Goodbye!
```

(Arjun issues the only copy; Ram and Ravi join the waitlist; the copy is held for Ram after the return; undo reverses the return)

```
Member view, keyword search and search by ID

  Issued "Data Structures Using C" to Arjun.
  Member 1: Arjun  (hash bucket 1)
  Books issued:
    - [110] Data Structures Using C
  Member 12: Ravi  (hash bucket 1)
  No books issued.
  ID    Title                          Author                Avail  Waiting
  ------------------------------------------------------------------------
  110   Data Structures Using C        Reema Thareja         2/3   0
  ID    Title                          Author                Avail  Waiting
  ------------------------------------------------------------------------
  104   Database System Concepts       Korth                 1/1   0
Goodbye!
```

(member view with hash bucket, keyword search and search by ID)